

递归循环 Transformer

Recurrent Looped Transformer

Yifan Zhang Jichen Feng Shihan Qin

yifanzhangresearch@gmail.com

2026 年 9 月 12 日*

摘要

递归循环 Transformer (Recurrent Looped Transformer, RLT) 将解码器的最终隐藏状态传递到下一个 token，并与该 token 的因果编码器表示一同作为输入。解码器还在每一层保留滑动窗口注意力 (sliding-window attention, SWA) 缓存。对于提示和回复中的每个 token，模型都会更新这两类状态，使递归计算跨越提示与回复的边界持续进行。沿连续状态展开的计算路径随序列长度增长，而每个 token 执行的模块数保持固定。本文说明如何批量处理编码器计算和独立序列的解码器更新，以及如何在强化学习 (RL) 中用当前参数重放已采样的历史。重放采用与采样相同的状态转移，并在参数更新后重建隐藏状态与缓存。与仅含解码器的 Transformer 相比，初步的八层模型实验显示：部分 RLT 层数分配更早学会奇偶校验，在无括号模运算上达到更高准确率；在带括号运算上的差距较小，在标准置换跟踪上的准确率均较低。奇偶校验采用三个初始化种子，其余任务采用一个。

项目主页: <https://github.com/yifanzhang-pro/recurrent-looped-transformer>

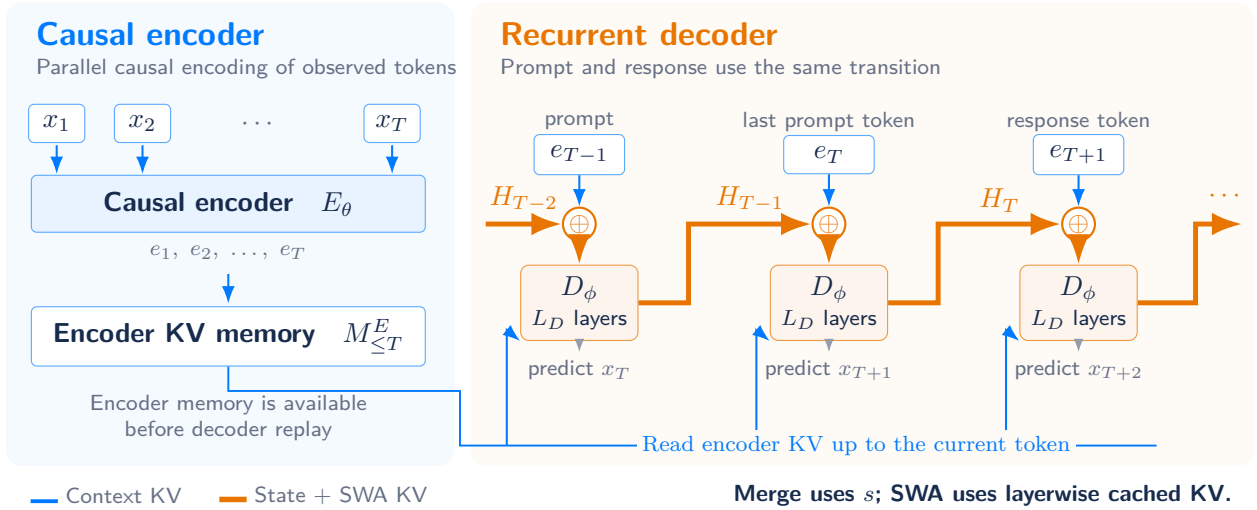


图 1 跨越提示与回复的递归计算。编码器构建因果键值 (KV) 记忆，解码器按 token 顺序处理。橙色箭头传递解码器状态 $H_t = (s_t, C_t^D)$: 融合操作读取 s_t ，每个 SWA 层读取各自的缓存。蓝色箭头提供截至当前位置的编码器记忆。图中展示提示的最后两次更新和回复的第一次更新。

*修订于 2026 年 9 月 15 日；中文译本对应同日英文公开版。

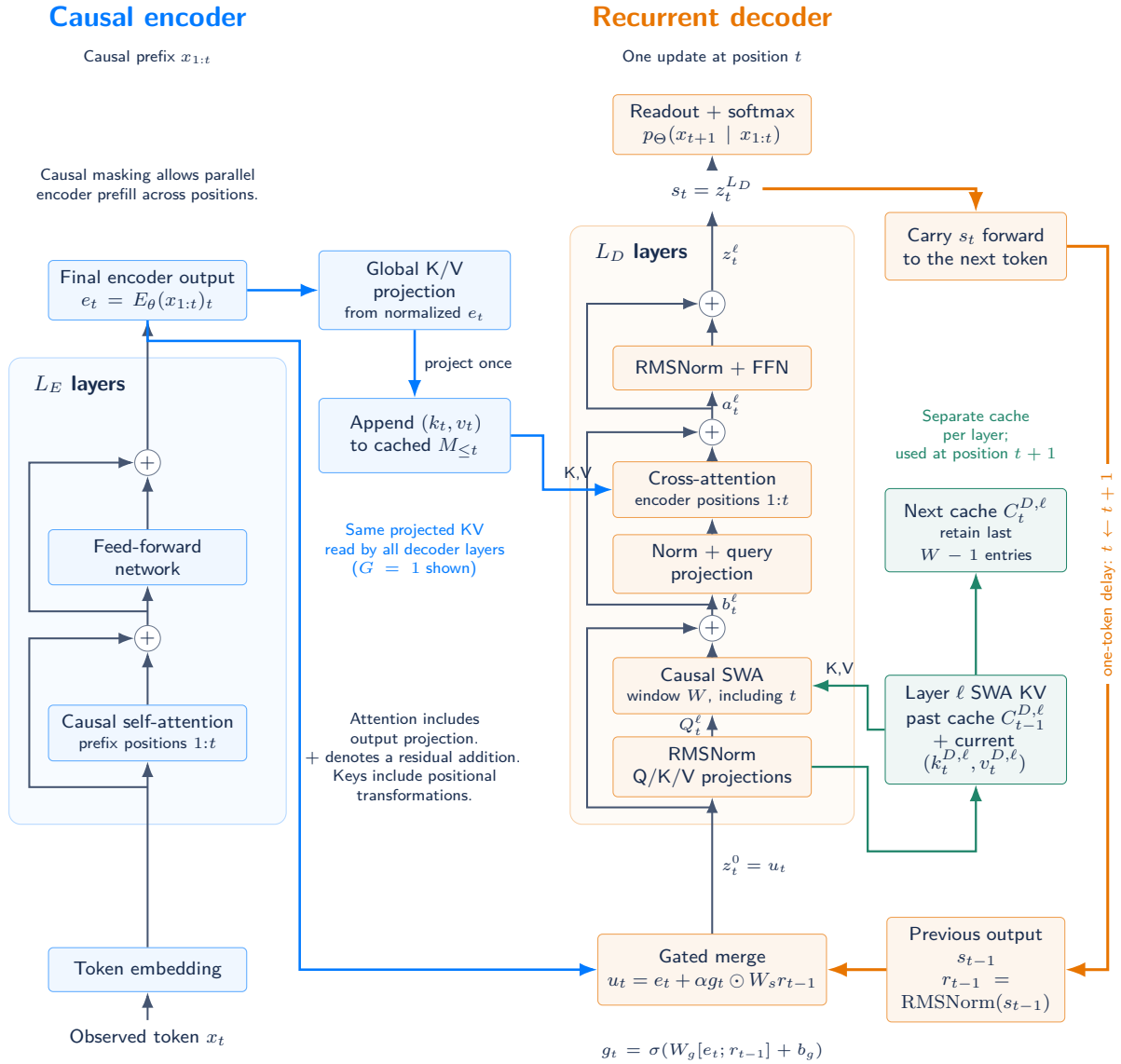


图 2 单次 token 更新的 RLTL 架构。计算沿编码器和解码器堆栈向上进行，各虚线框中的模块按标注的深度重复。编码器输出同时用于门控融合和全局 KV 投影。图中取 $G = 1$ ：所有解码器层使用各自的查询，读取同一份已投影并缓存的全局 KV。每个解码器层从自己的输入构建独立的 SWA KV，先关注局部窗口，再读取全局记忆，最后应用 FFN。残差路径绕过各注意力或 FFN 子层。解码器最终输出 s_t 进入下一个 token 的融合操作，各层同时保留自己的 SWA 缓存。蓝色箭头表示编码器特征和全局记忆，绿色表示局部 KV，橙色表示递归隐藏状态。

1 引言

递归循环 Transformer (RLT) 将因果编码器与由 Transformer 模块组成的递归解码器结合起来。编码器根据 token 历史构建记忆。在每个位置，解码器将当前编码器表示与自身上一步的输出融合，关注编码器记忆和近期解码器激活，并预测下一个 token。提示与回复使用同一套更新。

这一设计在 token 之间传递连续隐藏状态。处理 t 个 token 后，递归路径穿过 tL_D 个解码器模块，其中 L_D 为解码器深度。因此，序列越长，计算路径越长，而无需增加可学习的层。编码器和解码器中相容的权重也可以共享。例如，一个共享权重的 48 层编码器与 48 层解码器，对同一组注意力和前馈权重执行两次逻辑计算，每个 token 共执行 96 个模块。

递归会带来调度成本：解码器必须按位置顺序运行，提示预填充阶段也如此。编码器的位置可以并行处理，不同独立序列的解码器更新则可以组成同一批次。本文说明如何利用这一分工复用权重和记忆，并在训练中使用激活检查点。

训练与采样还必须采用一致的状态构建方式。对于 RL，这意味着在评估动作概率之前，先用当前参数重放提示和已采样的回复。采样器记录的概率仍描述产生这些动作的行为策略。本文明确给出所需的前向重放与梯度路径，使参数更新后的这一区别保持清晰。

RLT 借鉴了 YOCO 的编码器记忆复用机制 (Sun et al., 2024)，以及 Feedback Transformer、Recurrent Transformer、Full-bandwidth Transformer、T²MLR 和 Latent Recurrent Transformer 的时间反馈机制 (Fan et al., 2020; Oncescu et al., 2026; Wang et al., 2026; Cai et al., 2026; Huang et al., 2026)。本文关注贯穿整个解码器的反馈、相应的执行调度，以及它在策略重放中的使用。实验在加法、奇偶校验、模运算和置换状态跟踪任务上比较不同的八层配置。

2 方法

2.1 序列与状态

设 $x_{1:S}$ 为以 $x_1 = \text{BOS}$ 开始的一条独立序列。推理时， $x_{1:T}$ 为提示，后续 token 构成回复。模型在 T 两侧使用同一条件分布。令 L_E 和 L_D 分别为编码器与解码器深度， d 为残差宽度。编码器表示 $e_t \in \mathbb{R}^d$ 和递归输出 $s_t \in \mathbb{R}^d$ 均写作列向量。

完整的解码器状态为 $H_t = (s_t, C_t^D)$ 。缓存 C_t^D 保存各解码器 SWA 层保留的键值投影。窗口大小 $W \geq 1$ 包含当前 token，因此每层在一次更新后至多保留 $W - 1$ 个历史位置。所有参数（包括可学习初始状态 $s_\star$ ）统记为 Θ 。编码器和解码器分别记作 E_θ 和 D_ϕ 。

2.2 因果编码器与记忆

对于已观察到的前缀，计算

$$e_{1:T} = E_\theta(x_{1:T}). \quad (2.1)$$

每个编码器层内的位置可使用因果掩码一起处理，而编码器各层仍按深度顺序执行。对于解码器记忆组 $g \in \{1, \dots, G\}$ ，构造

$$k_t^g = \mathcal{P}_K^g(e_t, t), \quad v_t^g = W_V^g \text{RMSNorm}_E(e_t), \quad M_{\leq t}^g = \{(k_j^g, v_j^g)\}_{j=1}^t. \quad (2.2)$$

键映射包含归一化、投影以及所采用的位置变换。解码器第 ℓ 层读取组 $g(\ell)$ ： $G = 1$ 表示各层共享记忆， $G = L_D$ 则允许各层使用独立投影。解码器通过不同的注意力子层读取两类存储。交叉注意力读取覆盖整个前缀的编码器记忆 $M_{\leq t}$ ；因果 SWA 读取有界窗口内的解码器 KV。

2.3 每个 token 使用同一状态转移

在 BOS 之前仅初始化一次：

$$H_0 = (s_\star, \emptyset). \quad (2.3)$$

对每个已观察到或已采样的 token，执行

$$u_t = \text{Merge}(e_t, s_{t-1}), \quad (2.4)$$

$$H_t = (s_t, C_t^D) = D_\phi(u_t; M_{\leq t}, C_{t-1}^D, t), \quad t \geq 1, \quad (2.5)$$

$$p_\Theta(x_{t+1} | x_{1:t}) = \text{softmax}(W_o \text{RMSNorm}_o(s_t))_{x_{t+1}}. \quad (2.6)$$

对 $H = (s, C^D)$ ，定义 $F_t(H) = D_\phi(\text{Merge}(e_t, s); M_{\leq t}, C^D, t)$ 。编码器特征可以在解码器运行前计算。随后，为预测第一个回复 token，解码器必须处理提示中的每个位置：

$$H_T = F_T \circ F_{T-1} \circ \dots \circ F_1(H_0). \quad (2.7)$$

生成从 H_T 继续。根据 s_T 采样 x_{T+1} 后，增量编码该 token，追加由编码器生成的 KV，并计算 $H_{T+1} = F_{T+1}(H_T)$ ，其中包含各解码器 SWA 缓存的更新。在提示与回复的服务边界处， H_T 的两个分量都不重置。

2.4 提示预填充与增量解码

预填充先计算提示的因果编码器表示与记忆，再依次求出 $H_1, \dots, H_T$ 。即使整个提示的记忆已经可用，解码器在位置 t 的注意力仍限制为 $M_{\leq t}$ 。生成阶段使用

$$(e_t, C_t^E) = E_\theta^{\text{step}}(x_t, C_{t-1}^E) \quad (2.8)$$

更新编码器缓存 C^E ，随后追加记忆并更新解码器。若保留编码器缓存，已观察到的 token 可以分块编码。解码器仍逐 token 顺序处理：每次更新都产生后续位置需要的隐藏状态与 SWA KV。

2.5 融合与解码器模块

采用以下门控融合：

$$r_{t-1} = \text{RMSNorm}_s(s_{t-1}), \quad (2.9)$$

$$g_t = \sigma(W_g[e_t; r_{t-1}] + b_g), \quad (2.10)$$

$$u_t = e_t + \alpha g_t \odot W_s r_{t-1}. \quad (2.11)$$

其中 $W_g \in \mathbb{R}^{d \times 2d}$ ， $W_s \in \mathbb{R}^{d \times d}$ ， α 控制反馈幅度。实验使用 $\alpha = 0.1$ 。标量门控或低秩 W_s 可以降低融合成本。

从 $z_t^0 = u_t$ 开始，每个解码器模块依次应用因果 SWA、编码器记忆交叉注意力和前馈网络 (FFN)：

$$q_t^{D,\ell} = \mathcal{P}_Q^{D,\ell}(z_t^{\ell-1}, t), \quad (2.12)$$

$$k_t^{D,\ell} = \mathcal{P}_K^{D,\ell}(z_t^{\ell-1}, t), \quad v_t^{D,\ell} = W_V^{D,\ell} \text{RMSNorm}_{S,\ell}(z_t^{\ell-1}), \quad (2.13)$$

$$b_t^\ell = z_t^{\ell-1} + \text{Attn}_\ell^D(q_t^{D,\ell}, \{(k_j^{D,\ell}, v_j^{D,\ell})\}_{j=\max(1, t-W+1)}^t), \quad (2.14)$$

$$a_t^\ell = b_t^\ell + \text{Attn}_\ell^M(\mathcal{P}_Q^{M,\ell}(b_t^\ell, t), M_{\leq t}^{g(\ell)}), \quad (2.15)$$

$$z_t^\ell = a_t^\ell + \text{FFN}_\ell(\text{RMSNorm}_{D,\ell}(a_t^\ell)), \quad s_t = z_t^{L_D}. \quad (2.16)$$

注意力算子包含输出投影；查询与键映射包含各自的归一化和位置变换。各层在执行 SWA 前，先由该层输入形成当前位置的 KV，因此关注当前位置不会引入循环依赖。历史解码器 KV 来自 C_{t-1}^D 。更新后，在 C_t^D 中保留位置 $\max(1, t - W + 2), \dots, t$ ；当 $W = 1$ 时，该集合为空。预填充、采样和重放采用相同的位置元数据约定。不同的子层顺序定义不同的变体，必须在所有执行模式中保持一致。

图 2 展开了单次 token 更新中的编码器与解码器堆栈。

2.6 编码器与解码器的权重共享

共享权重的参考 RLT 令 $L_E = L_D = L$ 。编码器第 ℓ 层的自注意力与解码器第 ℓ 层的 SWA 共享相容的查询、键、值和输出投影矩阵，FFN 也共享权重。编码器使用自身的因果上下文，解码器 SWA 使用窗口内的解码器激活。解码器交叉注意力拥有独立的查询及输出投影，并使用式 (2.2) 的记忆投影。编码器与解码器保留各自的归一化模块，融合与读出也为独立模块。交叉注意力在共享的注意力和 FFN 之外增加计算。

两次计算共享权重，但产生独立的激活。不共享权重的 E_θ, D_ϕ 使用各自的参数，递归形式相同。两种版本都可以共享记忆组；各解码器层仍维护独立的 SWA 缓存。

3 计算性质

3.1 提示与回复的一致性

命题 3.1 (服务划分不变性). 固定一条独立序列的参数、token 序列、位置约定和初始状态。假设因果编码器的执行在数学上等价，SWA 窗口与缓存更新相同，解码器操作是确定性的。先批量执行编码器预填充、再递归更新解码器，与逐 token 增量处理该前缀，得到相同的状态和下一 token 分布。因此，对于固定的 token 历史，移动提示与回复的划分位置不会改变条件分布。

证明. 因果编码器的等价性保证两种调度具有相同的 e_t 和 $M_{\leq t}$ 。二者均从 $H_0 = (s_\star, \emptyset)$ 开始。若在 $t-1$ 时状态一致，式 (2.5) 在 t 时会对相同输入执行相同操作，因此二者在 t 时的状态仍一致。由于状态转移不依赖服务划分，归纳可得结论。 $\square$

3.2 预填充计算量与顺序深度

令 $C_E^{\text{pf}}(T)$ 为编码器预填充计算量， $C_M(T)$ 为记忆投影计算量， $C_D^{\text{step}}(t)$ 为一次解码器计算量：它读取 t 条编码器记忆、每层至多 W 个解码器位置，并包含融合开销。于是

$$C_{\text{prefill}}(T) = C_E^{\text{pf}}(T) + C_M(T) + \sum_{t=1}^T C_D^{\text{step}}(t). \quad (3.1)$$

对于稠密注意力及与模型宽度成比例的 KV 宽度，粗略的算术计算量为

$$O((L_E + L_D)(Td^2 + T^2d) + GTd^2 + L_D T \min(W, T)d). \quad (3.2)$$

预填充需要 T 次顺序执行的解码器状态转移，每次包含 L_D 个模块。即使算术复杂度与稠密 Transformer 同阶，这一依赖也可能降低硬件利用率。

每个新 token 执行 $L_E + L_D$ 个模块，另加融合与记忆投影。注意力计算量仍随上下文长度增长。设有效 KV 宽度为 d_{KV} ，则推理缓存的存储量近似为

$$O((L_E + G)t d_{\text{KV}} + L_D \min(t, W - 1)d_{\text{KV}}^D + d). \quad (3.3)$$

其中 d_{KV}^D 为解码器 SWA 的有效 KV 宽度， $O(d)$ 项用于保存当前递归输出。SWA 项只计算保留的历史；当前位置的 KV 和训练激活还需要额外存储。权重共享减少参数存储，但两次计算及其缓存仍不可省略。

3.3 递归路径深度

处理长度为 T 的提示和 n 个回复 token 后，递归路径已穿过 $(T+n)L_D$ 个解码器模块。图 3 将这条不断增长的路径与每 token 固定的模块数作比较。早期状态的影响取决于学到的门控、投影，以及状态转移雅可比矩阵的乘积。

Block count per token and along the recurrent path

Illustrative 48-layer encoder + 48-layer decoder; counts exclude merge and readout.

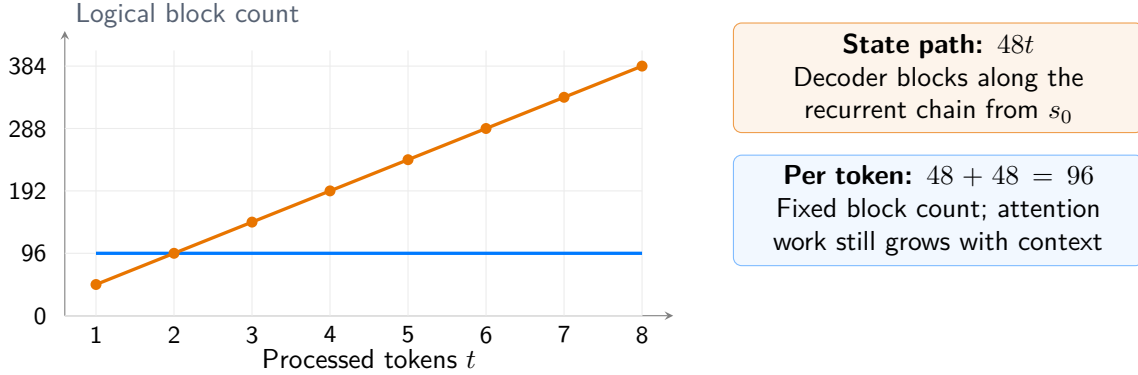


图 3 递归路径长度与每 token 的模块数。当 $L_E = L_D = 48$ 时，处理 t 个 token 后，路径穿过 $48t$ 个解码器模块。每个 token 执行 96 个编码器和解码器模块。橙色统计递归路径上的解码器模块，蓝色统计每个 token 在两个堆栈中执行的模块。

4 硬件执行

4.1 独立序列的批处理

对于已知的训练序列或提示，编码器可跨位置并行计算特征与记忆投影。解码器随后按位置顺序处理。如图 4 所示，不同独立序列的下一轮解码器更新可以组成同一批次。每条序列分别维护自己的编码器前缀、SWA 缓存、递归输出和位置。

推理时，每一步包含编码器更新、记忆追加、融合、解码器计算和读出。将请求组成批次可增大矩阵运算规模，并复用权重。小批量和不均匀的序列长度可能限制利用率。

4.2 内存访问与参数复用

当前缀和参数不变时，编码器 KV 保持不变。解码器各层复用该记忆，并将自己的 KV 追加到各自的 SWA 缓存中。较少的记忆组可减少 KV 存储和投影计算，较多的组则允许各层使用不同的变换。编码器与解码器共享权重也可能有助于让权重常驻加速器。

一种拟议的优化是融合归一化、门控、状态投影和残差相加，同时保持其依赖关系与数值行为。注意力内核必须保持编码器前缀掩码和局部 SWA 窗口。

Batching decoder updates from independent sequences

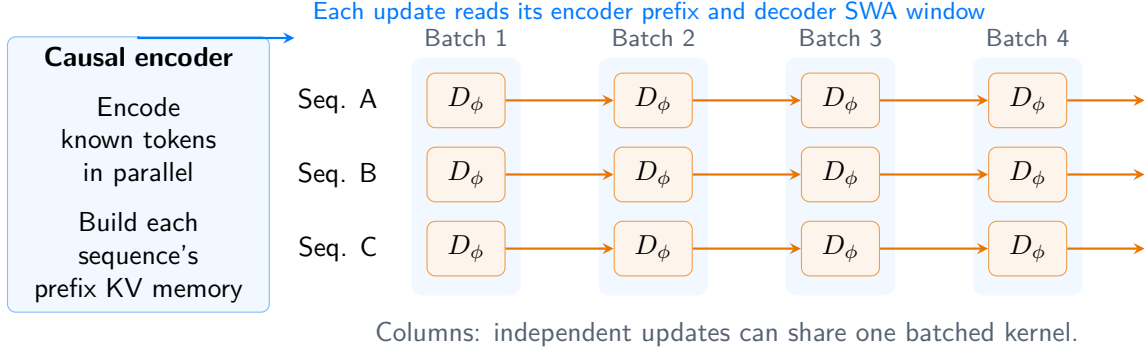


图 4 跨序列批量执行解码器更新。每列将独立更新组成一个批次，每行按 token 顺序展示一条序列。每次更新读取自己的编码器前缀和解码器 SWA 窗口。已知 token 可以在重放前并行编码，生成的 token 则在产生后编码。

4.3 训练内存与数值一致性

激活检查点通过在随时间反向传播（BPTT）时重算激活，减少需要保存的激活量。批量大小和检查点位置取决于序列长度及可用的加速器内存。完整 BPTT 保留贯穿整个递归历史的梯度。

Zhang et al. (2026) 指出，实际执行的策略不仅取决于权重，还取决于精度、缓存构造、归约运算和采样变换。因此，采样器与训练器的实现需要保持位置约定、随机行为和缓存内容的一致，并在不同执行模式之间进行数值检查。

5 训练目标

5.1 自回归预训练

基础目标是全序列的下一 token 预测：

$$\mathcal{L}_{\text{PT}}(\Theta) = -\mathbb{E}_{x_{1:S}} \left[\frac{1}{S-1} \sum_{t=1}^{S-1} \log p_{\Theta}(x_{t+1} \mid x_{1:t}) \right]. \quad (5.1)$$

初始化 $H_0 = (s_*, \emptyset)$ ，计算因果编码器特征，并在位置 $1, \dots, S-1$ 上更新解码器。每个非 BOS 目标都计入损失；编码器、记忆投影、融合和解码器联合训练。开始一篇独立文档时，重置解码器与编码器状态及位置，并禁止跨文档边界的注意力。分段训练时，应明确初始上下文与状态：丢弃较早的状态会改变似然所依赖的历史。

5.2 监督微调

对助手目标令 $m_{t+1} = 1$ ，对用户、系统、工具或填充目标令其为 0。对于至少包含一个选中目标的样本，优化

$$\mathcal{L}_{\text{SFT}}(\Theta) = -\mathbb{E}_x \left[\frac{1}{\sum_{t=1}^{S-1} m_{t+1}} \sum_{\substack{1 \leq t < S \\ m_{t+1}=1}} \log p_{\Theta}(x_{t+1} \mid x_{1:t}) \right]. \quad (5.2)$$

该掩码只选择损失项。用户、系统和工具 token 仍更新解码器状态，助手损失的梯度会穿过这些更新，包括编码器记忆与解码器 KV。助手回合开始时，模型从已有状态继续。

5.3 策略梯度与重放

令 $c = x_{1:T}$ 为提示， $y = (y_1, \dots, y_N)$ 为采样回复，其中 $y_i = x_{T+i}$ 。策略为

$$\pi_{\Theta}(y \mid c) = \prod_{i=1}^N p_{\Theta}(y_i \mid c, y_{<i}). \quad (5.3)$$

对于不依赖 Θ 的序列奖励 $R(c, y)$ ，定义 $J(\Theta) = \mathbb{E}_{c, y \sim \pi_{\Theta}}[R(c, y)]$ 。其同策略得分函数梯度为

$$\nabla_{\Theta} J = \mathbb{E}_{c, y \sim \pi_{\Theta}} \left[(R(c, y) - b(c)) \sum_{i=1}^N \nabla_{\Theta} \log p_{\Theta}(y_i \mid c, y_{<i}) \right], \quad (5.4)$$

其中 $b(c)$ 是不依赖回复的基线，在计算策略梯度时视为常数。重放时，已采样的 token 固定，梯度穿过用于计算这些 token 对数概率的递归计算。

图 5 展示重放流程。采样器记录每个动作在实际采样分布 μ 下的对数概率。训练器从初始状态出发，在当前参数 Θ 下处理完整提示和已采样的回复前缀，重建编码器特征、递归输出及解码器 SWA 缓存。由所得动作概率构造比率

$$r_i(\Theta) = \exp(\log p_{\Theta}(y_i \mid c, y_{<i}) - \log \mu(y_i \mid c, y_{<i})). \quad (5.5)$$

此处目标策略为原始模型策略 p_{Θ} 。行为概率必须包含采样器使用的温度缩放、截断和重新归一化。精确重要性采样要求 $p_{\Theta}(\cdot \mid h) \ll \mu(\cdot \mid h)$ ：在每个相关历史下，目标概率为正的动作，其行为概率也必须为正。Top- k 或 top- p 采样通常会排除未经截断的 softmax 目标中概率为正的动作。如果目标策略本身也使用采样变换，应在目标函数与比率中一致应用该变换。

训练器必须同时保持前向概率及其对参数的导数 (Zhang et al., 2026)。因此，它需要对递归计算或其等价实现求导。

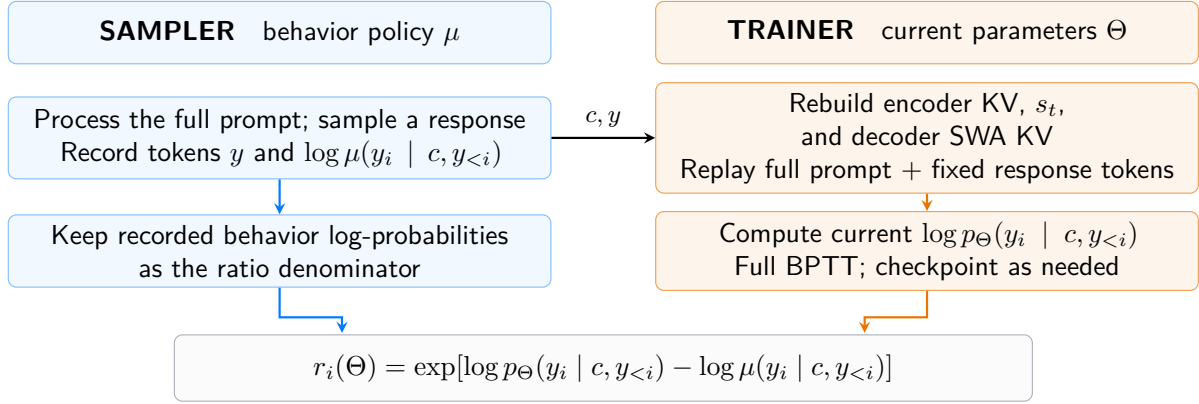
对于固定提示和序列奖励，精确的轨迹重要性采样使用 $\prod_i r_i$ ，并需满足支持集与可积性条件。即使重放概率正确，逐 token 裁剪等替代损失仍可能引入偏差。

5.4 训练中的梯度与缓存复用

完整 BPTT 沿整个提示和回复传播梯度，路径包括编码器记忆、递归输出和解码器 SWA KV。即使仅回复 token 贡献损失，这一点也不变。在当前参数下用 `no_grad` 重算提示，可得到相同的前向值，但会丢失提示状态的导数。

参考计算采用完整 BPTT，并按需使用激活检查点。检查点机制在反向传播时以相同参数和随机状态重算激活。截断 BPTT (TBPTT) 则在边界处分离选定张量，保留其数值但切断梯度路径。仅分离 s_t 仍会留下经过解码器 KV 和编码器记忆的路径，因此截断方案必须说明分离哪些张量。

Replaying a sampled history under current parameters



After every parameter update: rebuild current-policy KV and states; retain original behavior log-probabilities.

图 5 在当前参数下重放采样历史。采样器记录行为概率。训练器重建完整前缀，包括递归输出和解码器 SWA 缓存，以评估当前策略。比率比较当前动作概率与记录的行为概率。

每次重放及反向传播期间，参数保持固定。优化器更新后，在下次精确重放前，重建编码器 KV、递归输出和解码器 SWA KV。重算可从序列起点开始，也可从使用当前参数与执行设置生成的精确前缀检查点开始。已记录的行为对数概率保持不变，因为它们描述的是采样这些动作时的策略。

6 多轮推理与缓存复用

对话构成一条从 BOS 开始的 token 历史。用户消息、工具结果、角色分隔符和助手 token 都执行编码器与解码器更新。例如，工具返回时，先结合缓存前缀编码其输出，再将这些 token 依次送入解码器，随后恢复生成。仅在开始新的独立序列或明确执行上下文重置时重置状态。

若需从保存的前缀继续，应保留编码器缓存 C_t^E 、交叉注意力记忆 $M_{\leq t}$ 、解码器状态 $H_t = (s_t, C_t^D)$ 、token 与位置、SWA 窗口约定以及模型版本。权重固定时，无论提示在哪个位置结束，这些值都可复用。若要复现相同的采样输出，还需要保存采样器的随机状态。如果只保存了文本，应重放文本来重建状态。

编辑或截断较早的 token 会使后续缓存状态失效。应从编辑位置之前的有效检查点重新计算；改写早期 token 的聊天模板也遵循同一规则。在多层 RL 中，用户和工具 token 更新状态并传递梯度，但不乘动作重要性比率，因为这些 token 并非由策略采样。

7 初步算法任务实验

我们在加法、奇偶校验、模运算和置换状态跟踪任务上比较不同的编码器与解码器深度分配。结果来自 2026 年 9 月 15 日 16:11:47 至 16:12:10 UTC 采集的快照，当时训练仍在进行。截至该时刻，48 次运行中有 21 次已达到计划的 2,000 个优化器步骤。所有分数均来自训练长度范围内的独立验证样本。

表 1 各任务所有运行共同达到的最新步骤上的验证准确率 (%)。同一行中的六个模型已经使用相同数量的训练样本。奇偶校验报告三个初始化种子的均值 $\pm$ 样本 SD，其余项使用一个种子。

Task	Step	RLT 4+4	RLT 5+3	RLT 6+2	RLT 7+1	RLT 8+0	GPT 8
Addition	1000	100.00	98.44	100.00	100.00	100.00	100.00
Parity	500	83.29 $\pm$ 28.94	83.51 $\pm$ 28.00	99.44 $\pm$ 0.98	82.77 $\pm$ 29.84	48.70 $\pm$ 2.60	48.48 $\pm$ 0.53
Mod 5, no brackets	800	22.40	21.74	20.83	22.53	21.74	20.70
Mod 5, brackets	800	37.50	36.72	36.72	40.10	37.63	35.55
S_5 , swaps	1000	100.00	99.61	95.70	89.84	86.33	89.84
S_5 , standard	1000	1.17	3.52	1.95	1.56	1.17	1.17

7.1 模型、任务与评估协议

所有模型均有八个逻辑层，宽度为 512，FFN 宽度为 1,365，注意力头数为四。我们将不共享权重的 RLT 4+4、5+3、6+2、7+1 和 8+0 配置，与八层的仅含解码器的 Transformer 比较。RLT 使用大小为八的 SWA 窗口、一个共享编码器记忆组，以及反馈幅度 $\alpha = 0.1$ 。8+0 变体没有解码器模块，但仍执行门控递归融合。RLT 的参数量为 26.10-28.73M，Transformer 为 25.31M (表 3)。

快照采集之后，我们实现了一个 4+4 的无隐藏状态反馈 RLT 对照模型 (RLT w/o feedback)，用于后续消融实验。它移除隐藏状态反馈，保留编码器、解码器 SWA 和限制为前缀的记忆注意力。对于已知 token，各解码器层内的位置可以并行处理；生成仍为自回归过程。图 6 展示其架构以及已知 token 的并行执行方式。附录 D 给出配置与实现细节。

所有运行使用全局批量 512、微批量 32，以及 $(\beta_1, \beta_2) = (0.9, 0.95)$ 的 AdamW；权重衰减为 0.1，梯度范数裁剪阈值为 1。基础学习率在前 200 步升至 10^{-4} ，随后按余弦衰减，在第 2,000 步降至 5×10^{-6} 。两类架构采用相同的宽度 μP 配方，以四个线程在 CPU FP32 下运行。RLT 的 TBPTT 间隔为 128 个 token，覆盖这些任务的每条训练序列，因此本实验中没有截断梯度。每 100 步执行一次验证。每次运行的完整预算为 1,024,000 个训练样本。

- **加法 (Addition):** 随机采样 1-8 位操作数，将数字按逆序序列化。256 个验证样本使用相同的位数范围。仅当贪心生成得到完整正确的答案和 EOS 时，才计为正确。
- **奇偶校验 (Parity):** 使用长度为 3-40 的二进制串训练，只监督最终的奇偶位。验证集汇总长度为 32、36 和 40 的样本，每种长度 256 个。
- **模运算 (Mod-5):** 计算由 +、- 和 $\times$ 构成的表达式模五的值。无括号变体遵循乘法优先级，训练表达式长度为 3-39 的奇数，验证长度为 33、35 和 39。带括号变体使用表达式树，训练长度为 3-40，验证长度为 32、36 和 40。每种验证长度包含 256 个样本，仅监督最终值。
- **S_5 状态跟踪:** 复合 32 个置换，并在每次输入后监督累计乘积，遵循 [Grazzi et al. \(2024\)](#)。标准变体的输入覆盖全部 120 个置换；交换变体使用恒等置换和十个单次对换。每个变体有 256 条长度为 32 的验证序列。主要指标为最终状态准确率，前缀准确率列于附录。

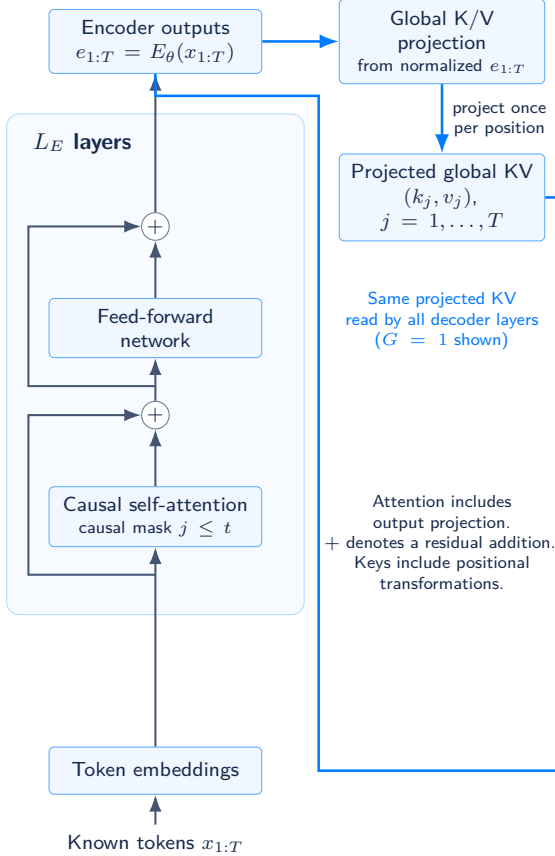
奇偶校验使用初始化种子 42、43 和 44，其余任务使用种子 42。各模型及各次奇偶校验运行的数据生成种子、训练样本和验证集均固定。对于奇偶校验，我们先合并每次运行中的验证样本，再报告三个初始化结果的均值与样本标准差 (SD)。该 SD 衡量固定数据流上由初始化引起的变异。

图 7 展示已取得的验证检查点。每个奇偶校验均值均使用同一步的全部三个种子；任一种子缺少检查点时，均值曲线即停止。表 1 在每个任务所需的所有运行共同达到的最新步骤上比较六个模型。表 2 单独比较已完成的模五运算运行在第 2,000 步的结果。

Causal encoder

Known sequence $x_{1:T}$

Causal masking allows parallel encoder prefill across positions.



Decoder w/o feedback

Parallel positions within each layer

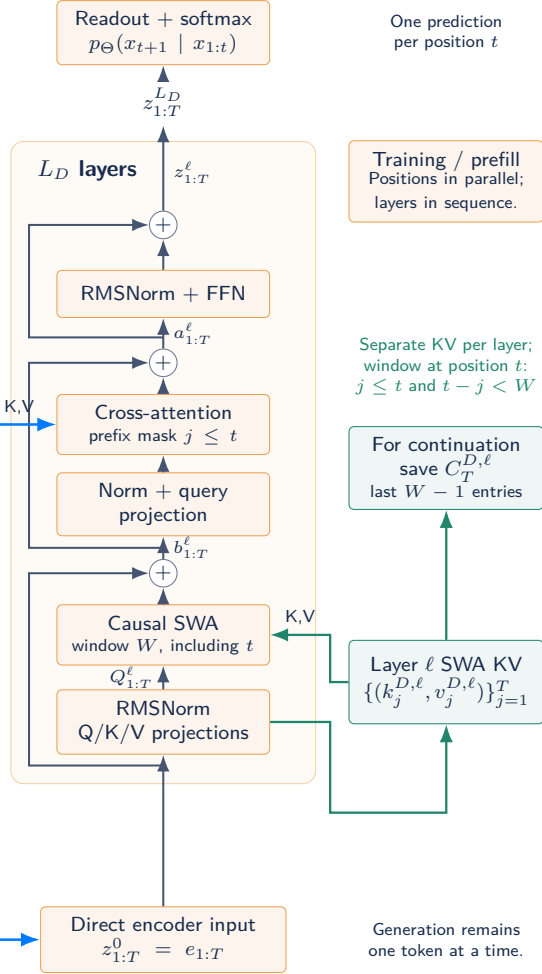


图 6 训练或提示预填充阶段的 RLT w/o feedback。解码器直接接收编码器输出， $z_{1:T}^0 = e_{1:T}$ ，门控融合与最终隐藏状态反馈路径均被移除。各虚线框中的模块按标注深度重复。当 $G = 1$ 时，所有解码器层读取同一份已投影的全局 KV；查询位置 t 可读取编码器位置 $j \leq t$ 。各解码器层构造自己的 SWA KV，并使用包含当前位置、大小为 W 的因果窗口。已知位置在每层内一起处理，各层则按深度顺序执行。增量生成保留编码器缓存、全局记忆和各层最近的 $W - 1$ 条 SWA 缓存，逐 token 进行。

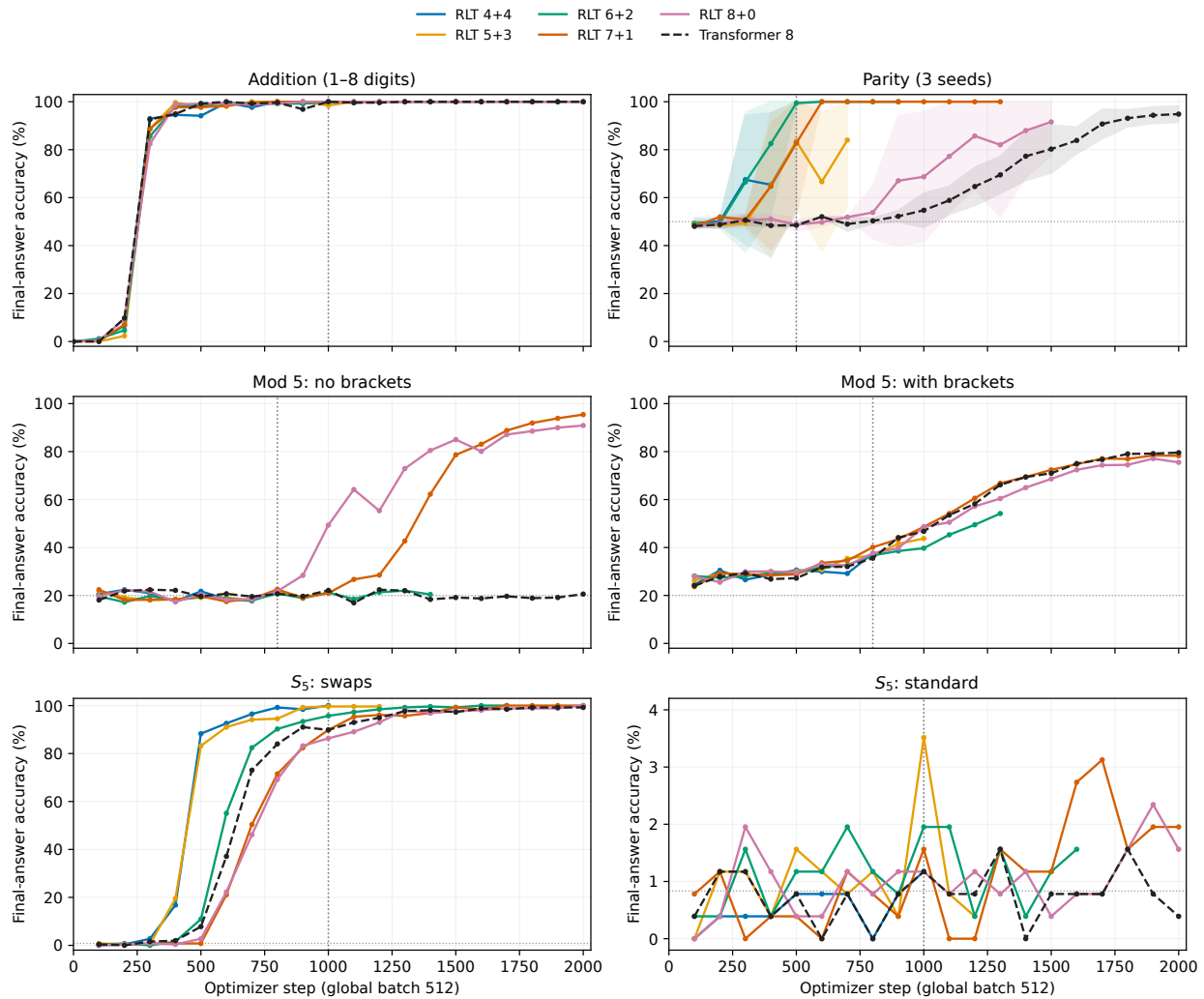


图 7 训练过程中的验证准确率。加法要求贪心生成的答案和 EOS 全部正确；其余任务测量最终答案或最终状态准确率。奇偶校验在每个绘制的步骤上显示三个初始化种子的均值 $\pm$ 样本 SD ($n = 3$)，阴影带裁剪到准确率取值范围内。其余面板显示单个种子。曲线未经平滑，并终止于可用检查点。竖向点线标记表 1 所用步骤，横向点线表示均匀预测的准确率。标准 S_5 的准确率较低，因此使用较窄的纵轴范围。

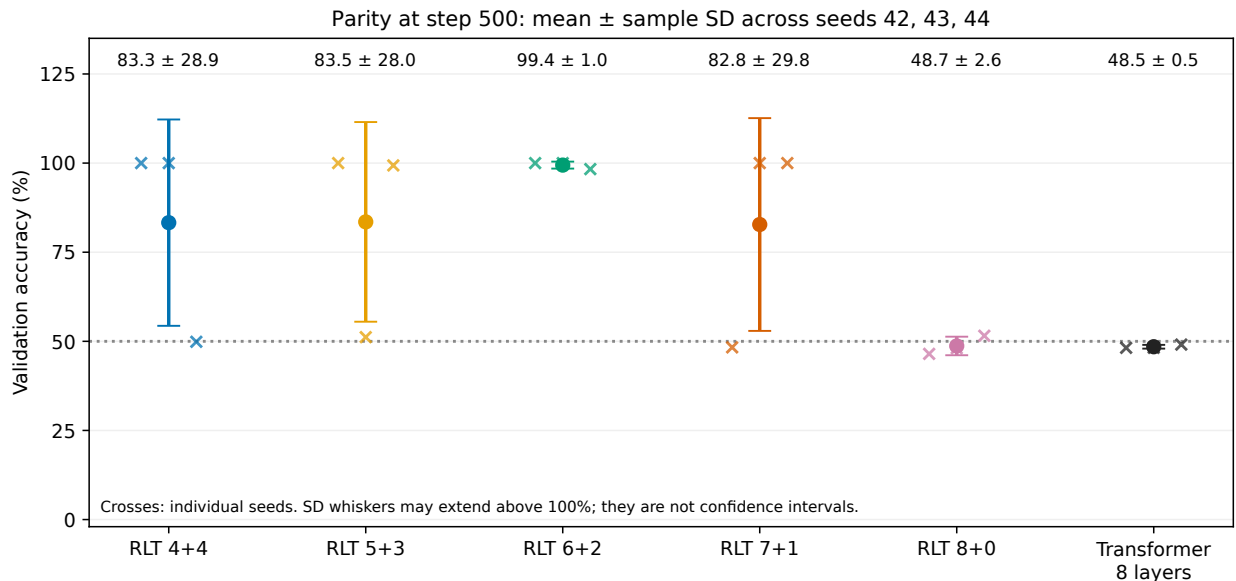


图 8 训练 500 步后的奇偶校验结果。圆点和误差棒显示种子 42、43 和 44 的均值与样本 SD；叉号表示单个种子，并作少量水平偏移以便辨认。SD 误差棒描述种子间的离散程度，可能超过 100%。所有模型使用相同的 768 个验证样本，每个种子均已使用 256,000 个训练样本。

表 2 已完成的模五运算符集：第 2,000 步，种子 42。各项为使用 1,024,000 个训练样本后的验证准确率 (%)。采集快照时，这三个模型均已完成两种变体的训练，分配更多解码器层的配置仍在运行。

Task	RLT 7+1	RLT 8+0	GPT 8
No brackets	95.44	90.89	20.57
With brackets	78.26	75.52	79.56

7.2 奇偶校验：学习速度与初始化变异

在第 500 步，RLT 6+2 的验证准确率为 $99.44 \pm 0.98\%$ ，Transformer 为 $48.48 \pm 0.53\%$ 。RLT 4+4、5+3 和 7+1 的均值约为 83%，SD 为 28–30 个百分点；8+0 仍接近随机水平。图 8 展示这些均值对应的各个种子结果。由于验证集固定，其离散程度反映不同初始化产生的训练结果差异。

RLT 6+2 的三个种子均在第 600 步前（含该步）达到 100%。在这些运行中，Transformer 学习较慢，在第 2,000 步达到 $94.84 \pm 3.43\%$ 。

7.3 带括号与无括号模运算

第 800 步时，六个模型在无括号模运算上均接近 20% 的均匀预测参考值。使用种子 42 训练至第 2,000 步时，RLT 7+1 和 8+0 分别达到 95.44% 和 90.89%，Transformer 仍为 20.57%（表 2）。

在带括号表达式上，Transformer 达到 79.56%，上述两种 RLT 配置分别为 78.26% 和 75.52%。两种任务生成器的运算符结构和标签分布不同。括号也占据位置，因此相同的 token 长度可能包含不同数量的算术运算。

7.4 加法与置换状态跟踪

第 1,000 步时, 除 RLT $5 + 3$ 为 98.44% 外, 所有模型的加法准确率均达到 100%。因此, 在该检查点及 1–8 位的训练范围内, 这项任务对模型的区分度较小。

同一步骤的交换 S_5 任务中, RLT $4 + 4$ 和 $5 + 3$ 的最终状态准确率分别为 100% 和 99.61%, Transformer 为 89.84%。Transformer 的前缀 token 准确率已达 98.75%, 因此对前缀求平均会掩盖部分序列末尾的错误。到第 2,000 步, 已完成的 $7 + 1$ 、 $8 + 0$ 和 Transformer 运行的最终状态准确率分别达到 100%、100% 和 99.22%。

标准 S_5 对六个模型都较困难: 第 1,000 步的分数为 1.17–3.52%, 120 类均匀预测的参考值为 0.83%。

7.5 训练稳定性与比较范围

奇偶校验中, 种子 42 的 RLT $5 + 3$ 从第 500 步的 100% 降至第 600 步的 48.05%, 随后在第 700 步恢复至 100%。图 9 展示训练损失, 附录 D 给出奇偶校验的各种子曲线。这类回退需要在更多种子上开展学习率和反馈消融; 统一的优化器配方对各架构的适配程度可能不同。

表现较好的编码器与解码器深度分配随任务而变。当前比较同时改变反馈、注意力结构、参数量和计算量, 因此需要匹配的 RLT w/o feedback 运行来分离隐藏状态反馈的作用。评估覆盖训练长度上的学习: 奇偶校验使用三个初始化, 其余各任务使用一个。硬件吞吐量与 RL 性能尚待测量。

8 相关工作

Transformer 与 RNN 混合模型。 Chen et al. (2018) 将 Transformer 编码器与基于 LSTM 的 RNMT+ 解码器结合, 用于机器翻译。RLT 同样采用并行编码与递归解码的总体分工。其解码器由带编码器记忆交叉注意力和 SWA 的 Transformer 模块组成, 并在因果语言模型中处理提示与回复的全部 token。

编码器生成的记忆。 YOCO 为上层交叉解码器构建可复用的 KV 记忆, 并允许预填充提前退出 (Sun et al., 2024)。DeepSeek-V4.1-Flash 从最终编码器状态投影得到解码器的全局 KV, 并维护独立的解码器 SWA (DeepSeek-AI, 2026)。RLT 也从编码器状态构建全局记忆, 但其递归解码器必须处理每个提示 token。

时间反馈。 Feedback Transformer 使后续计算能够访问过去的表示 (Fan et al., 2020)。Recurrent Transformer 根据每层的输出构建该层的持久 KV, 并给出精确的分块调度来改善内存数据搬运 (Oncescu et al., 2026)。RLT 将上一个 token 的最终解码器输出反馈到下一个 token 的解码器输入。它的全局记忆来自编码器, 局部 SWA KV 来自解码器。RLT 的递归依赖贯穿整个解码器, 而 Recurrent Transformer 的反馈发生在各层内部。

跨 token 的潜在状态反馈。 Full-bandwidth Transformer 通过门控线性单元, 将上一个 token 的顶层隐藏状态与下一个 token 的嵌入结合, 同时保留 Transformer 堆栈与 KV 缓存 (Wang et al., 2026)。其多遍训练在各遍之间平移隐藏状态, 以支持 token 并行的教师强制训练, 并混入前缀计算以处理提示与生成之间的差异。RLT 将这种输出到输入的反馈放在编码器与解码器接口处, 训练时在完整历史上顺序重放解码器。

T^2MLR 将上一个 token 缓存的中间层表示反馈到当前位置的较早层 (Cai et al., 2026)。其实实验发现, 中间层之间的递归可能优于贯穿整个网络的递归。训练通过固定次数的 Jacobi 迭代近似时间状态, 并单独控制反向传播深度。RLT 使用贯穿整个解码器的反馈, 以完整历史重放和完整 BPTT 为参考计算, 也可选用 TBPTT。

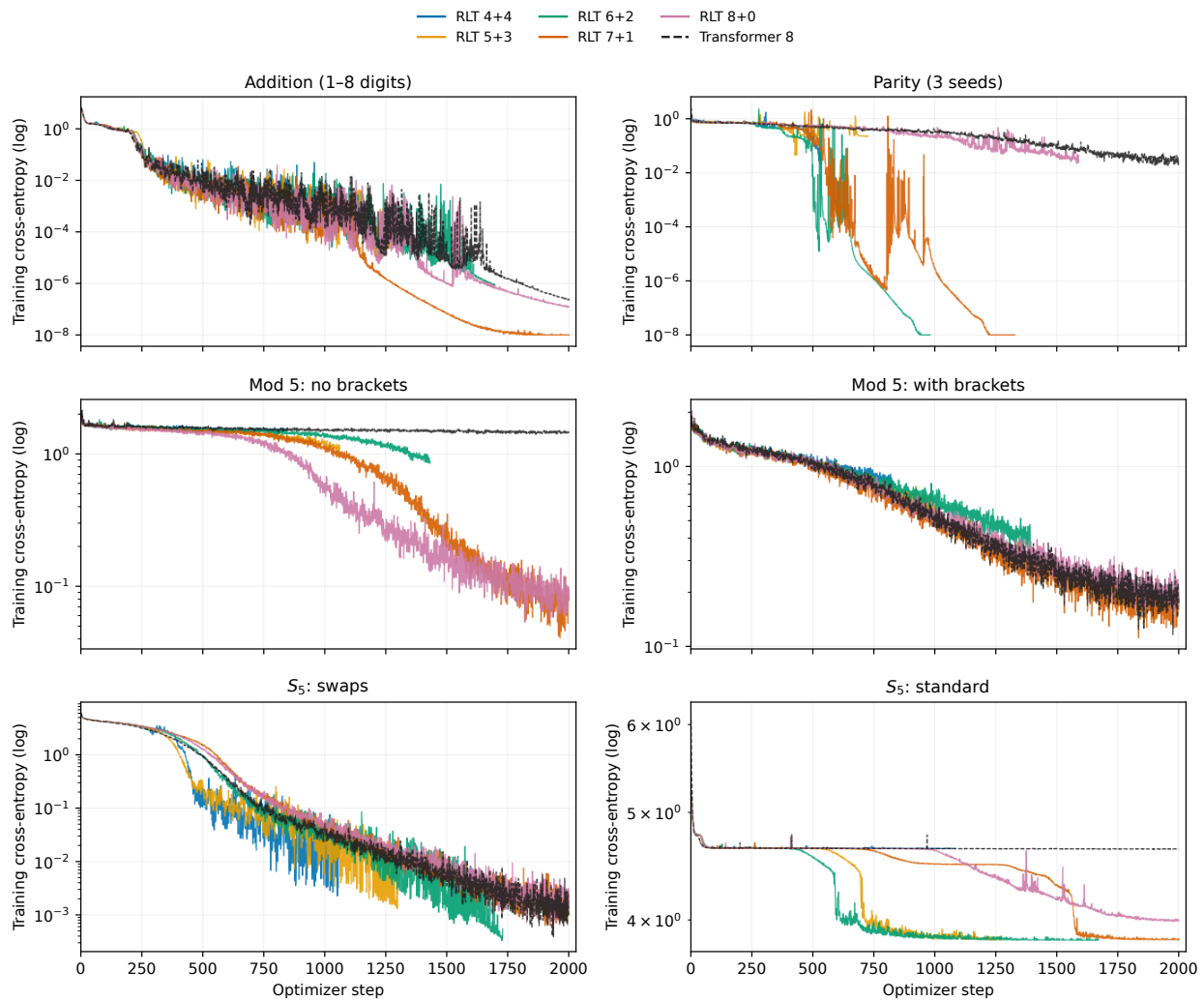


图 9 每个记录步骤上未经平滑的训练交叉熵。奇偶校验在每个可用的共同步骤上对全部三个种子求平均，其余任务展示种子 42。在对数轴上，低于 10^{-8} 的值按 10^{-8} 显示。损失在优化器更新前按全局批量计算，记录的梯度范数为裁剪前的值。各任务的监督方式不同，应在同一任务内比较损失。

潜在递归语言模型。 Latent Recurrent Transformer (LRT) 通过 KV 投影和残差注入复用上一个 token 的高层状态，保留仅含解码器的主干，并为每个生成 token 执行一次前向计算 (Huang et al., 2026)。其交错并行训练从共享状态缓冲区中读取状态，逐步细化部分位置。在 RL 中，它使用已分离梯度的 rollout 状态初始化该缓冲区，以减小 rollout 与重算之间的差异。RLT 将编码器记忆与递归解码器分开，并在当前参数下重建完整历史，以执行精确的策略重放。

连续潜在计算。 Coconut 在潜在推理阶段将最后的隐藏状态作为下一个输入嵌入反馈，并通过课程学习逐渐以连续状态替代文本推理步骤 (Hao et al., 2024)。PonderLM-2 在预训练期间于普通 token 之间插入潜在步骤，并使用 Jacobi 迭代并行近似这些步骤的递归依赖 (Zeng et al., 2025)。RLT 对提示或回复中的每个普通 token 更新一次状态，不增加潜在位置。

块级与段级递归。 Block-Recurrent Transformers 通过注意力和门控更新持久状态向量，并在每个递归步骤中并行处理一块 token (Hutchins et al., 2022)。Recurrent Memory Transformer 在序列段之间传递可学习记忆 token 的表示 (Bulatov et al., 2022)。RLT 在每个 token 处更新最终解码器输出和 SWA 缓存，同时维护独立的编码器记忆。这种更细的更新间隔要求在每个块内顺序执行解码器。

跨深度的权重共享。 Universal Transformers 在深度方向共享权重 (Dehghani et al., 2018)；递归深度潜在推理则重复应用递归模块以增加计算量 (Geiping et al., 2025)。RLT 跨 token 推进状态，并可选择共享编码器与解码器中相容的权重。

9 结论

RLT 在提示与回复的每个 token 之间传递解码器隐藏状态和 SWA 缓存。其递归路径随序列增长，编码器记忆则可独立构建并复用。同一状态转移定义训练、采样以及当前参数下的重放。

八层模型实验显示，部分编码器与解码器层数分配更早学会奇偶校验，并在单种子的无括号模运算实验中取得较大的准确率差距。不同层数分配的相对表现随任务而变。后续实验将在匹配的训练设置下比较 RLT 与 RLT w/o feedback，以分离隐藏状态反馈的作用。

参考文献

- Aydar Bulatov, Yuri Kuratov, and Mikhail S. Burtsev. Recurrent memory transformer. *arXiv preprint arXiv:2207.06881*, 2022. URL <https://arxiv.org/abs/2207.06881>.
- Ziyang Cai, Xingyu Zhu, Yihe Dong, Yinghui He, and Sanjeev Arora. T²MLR: Transformer with temporal middle-layer recurrence. *arXiv preprint arXiv:2607.15178*, 2026. URL <https://arxiv.org/abs/2607.15178>.
- Mia Xu Chen, Orhan Firat, Ankur Bapna, Melvin Johnson, Wolfgang Macherey, George Foster, Llion Jones, Niki Parmar, Mike Schuster, Zhifeng Chen, Yonghui Wu, and Macduff Hughes. The best of both worlds: Combining recent advances in neural machine translation. *arXiv preprint arXiv:1804.09849*, 2018. URL <https://arxiv.org/abs/1804.09849>.
- DeepSeek-AI. DeepSeek-V4.1-Flash: Pushing the limits of KV cache compression. Technical report, DeepSeek-AI, 2026. URL https://huggingface.co/deepseek-ai/DeepSeek-V4.1-Flash/resolve/main/DeepSeek_V41_Tech_Report.pdf. Sections 2.2 and 3.2.2; publicly available from the official DeepSeek model repository.

- Mostafa Dehghani, Stephan Gouws, Oriol Vinyals, Jakob Uszkoreit, and Łukasz Kaiser. Universal transformers. *arXiv preprint arXiv:1807.03819*, 2018. URL <https://arxiv.org/abs/1807.03819>.
- Angela Fan, Thibaut Lavril, Edouard Grave, Armand Joulin, and Sainbayar Sukhbaatar. Addressing some limitations of transformers with feedback memory. *arXiv preprint arXiv:2002.09402*, 2020. URL <https://arxiv.org/abs/2002.09402>.
- Jonas Geiping, Sean McLeish, Neel Jain, John Kirchenbauer, Siddharth Singh, Brian R. Bartoldson, Bhavya Kailkhura, Abhinav Bhatele, and Tom Goldstein. Scaling up test-time compute with latent reasoning: A recurrent depth approach. *arXiv preprint arXiv:2502.05171*, 2025. URL <https://arxiv.org/abs/2502.05171>.
- Riccardo Grazi, Julien Siems, Arber Zela, Jörg K. H. Franke, Frank Hutter, and Massimiliano Pontil. Unlocking state-tracking in linear rnns through negative eigenvalues. *arXiv preprint arXiv:2411.12537*, 2024. URL <https://arxiv.org/abs/2411.12537>.
- Shibo Hao, Sainbayar Sukhbaatar, DiJia Su, Xian Li, Zhiting Hu, Jason Weston, and Yuandong Tian. Training large language models to reason in a continuous latent space. *arXiv preprint arXiv:2412.06769*, 2024. URL <https://arxiv.org/abs/2412.06769>.
- Zeyi Huang, Xuehai He, Liliang Ren, Yiping Wang, Baolin Peng, Hao Cheng, Shuohang Wang, Pengcheng He, Jianfeng Gao, Yong Jae Lee, and Yelong Shen. Latent recurrent transformer: Architecture exploration, training strategies, and scaling behavior. *arXiv preprint arXiv:2605.26797*, 2026. URL <https://arxiv.org/abs/2605.26797>.
- DeLesley Hutchins, Imanol Schlag, Yuhuai Wu, Ethan Dyer, and Behnam Neyshabur. Block-recurrent transformers. *arXiv preprint arXiv:2203.07852*, 2022. URL <https://arxiv.org/abs/2203.07852>.
- Costin-Andrei Oncescu, Depen Morwani, Samy Jelassi, Alexandru Meterez, Mujin Kwun, and Sham Kakade. The recurrent transformer: Greater effective depth and efficient decoding. *arXiv preprint arXiv:2604.21215*, 2026. URL <https://arxiv.org/abs/2604.21215>.
- Yutao Sun, Li Dong, Yi Zhu, Shaohan Huang, Wenhui Wang, Shuming Ma, Quanlu Zhang, Jianyong Wang, and Furu Wei. You only cache once: Decoder-decoder architectures for language models. *arXiv preprint arXiv:2405.05254*, 2024. URL <https://arxiv.org/abs/2405.05254>.
- Xi Wang, Ziyang Cai, Zheng Zhan, Harry Dong, Ying Fan, Gustavo de Rosa, Tim Pearce, and John Langford. Full-bandwidth transformer. *arXiv preprint arXiv:2608.08888*, 2026. URL <https://arxiv.org/abs/2608.08888>.
- Boyi Zeng, He Li, Shixiang Song, Yixuan Wang, Ziwei He, Xinbing Wang, and Zhouhan Lin. PonderLM-2: Pretraining LLM with latent thoughts in continuous space. *arXiv preprint arXiv:2509.23184*, 2025. URL <https://arxiv.org/abs/2509.23184>.
- Yifan Zhang et al. Reliable RL scaling requires accounting for Prefill–Decode kernel mismatch. Technical report, Pretraining-RL-Science project, August 2026. URL https://github.com/yifanzhang-pro/Pretraining-RL-Science/blob/master/Prefill_Decode_Kernel_Mismatch.pdf. Dated August 6, 2026; revised August 24, 2026.

附录

A 执行流程	19
A.1 递归提示预填充	19
A.2 无隐藏状态反馈的 RLT	19
A.3 生成与新的外部输入	19
A.4 预训练与监督微调	19
A.5 当前策略的 RL 重放	20
B 因果性与梯度路径	20
C 缓存状态何时可以复用	21
D 实验细节	22
D.1 参数量	22
D.2 RLT w/o feedback 的实现	22
D.3 种子聚合与指标定义	22
D.4 快照复现	23

A 执行流程

完整的解码器状态为 $H_t = (s_t, C_t^D)$ ，初始值为 $H_0 = (s_*, \emptyset)$ 。编码器的续接状态与编码器生成的交叉注意力记忆分别维护。所有调度均使用式 (2.16) 中的模块顺序和窗口约定。

A.1 递归提示预填充

1. 以 $x_1 = \text{BOS}$ 和 $H_0 = (s_*, \emptyset)$ 开始一条独立序列，初始化编码器缓存与位置。
 2. 使用因果掩码并行编码 $x_{1:T}$ ，构建编码器 KV 记忆。
 3. 对 $t = 1, \dots, T$ ，计算 $H_t = D_\phi(\text{Merge}(e_t, s_{t-1}); M_{\leq t}, C_{t-1}^D, t)$ 。各解码器层读取当前位置的 KV 及保留的 SWA 历史。两种注意力掩码均排除 t 之后的位置。
 4. 从 s_T 预测第一个回复 token。为下一次更新保留 H_T 、编码器缓存、记忆和位置元数据。
- 每次解码器更新产生后续更新所需的 KV，因此即使编码器并行运行，这一循环仍为顺序执行。

A.2 无隐藏状态反馈的 RLT

RLT w/o feedback 移除最终解码器隐藏状态的反馈（图 6）。它保留因果编码器、编码器记忆、解码器 SWA 和限制为前缀的记忆注意力。可学习初始状态 s_* 、状态归一化、门控融合和反馈投影均被移除。每个解码器位置接收 e_t ，代替 $\text{Merge}(e_t, s_{t-1})$ 。

对于已知 token，解码器各位置在每层内使用因果 SWA 和编码器前缀掩码一起运行。所得各层 KV 可保存下来，用于后续增量续接。生成仍按顺序进行，因为未来 token 未知。全序列训练采用普通反向传播，不设置递归 TBPTT 边界。

该对照在保持编码器与解码器分工及注意力结构不变的条件下，检验隐藏状态反馈的作用。

A.3 生成与新的外部输入

从 s_t 预测的分布中采样 x_{t+1} ，随后处理该 token：更新编码器缓存与记忆，并将完整解码器状态推进到 H_{t+1} 。所得状态用于预测 x_{t+2} 。每个被处理的 token 恰好执行一次递归更新，并在每个解码器 SWA 层插入一次 KV。

当用户或工具提供多个 token 时，可在已有前缀条件下，使用因果批处理计算它们的编码器表示。解码器仍从保存的完整状态出发，按 token 顺序更新。外部 token 同时更新 s_t 和 C_t^D 。

若生成因长度上限而停止，且最后一个输出 token 尚未被模型处理，应单独记录该 token。缓存表示已经处理过的前缀；恢复时，先处理一次这个待处理 token，再处理后续 token。

A.4 预训练与监督微调

1. 用因果编码器计算 $e_{1:S-1}$ 并构建记忆。
2. 初始化 $H_0 = (s_*, \emptyset)$ ，展开位置 $1, \dots, S-1$ 的全部计算，包括各层 SWA 缓存更新。
3. 预训练累积所有有效的下一 token 损失，SFT 仅累积助手目标的损失。所有上下文 token 都执行状态更新。
4. 按每个样本选中的目标数量归一化，再按式 (5.1) 和式 (5.2) 对样本求平均。对完整计算反向传播，并按需使用检查点。

没有选中目标的样本不计入损失均值。若改为按 token 求平均，选中目标较多的样本会得到更高权重。

打包在一起的独立序列需要各自的递归状态、SWA 缓存、编码器缓存、位置和注意力掩码。即使损失被掩码屏蔽，两种注意力机制也必须限制在各自文档边界内。训练中的缓存操作必须保留梯度依赖，或支持等价的重算。

A.5 当前策略的 RL 重放

1. 读取 rollout 的 token 历史、动作掩码、实际行为对数概率与采样元数据。行为概率包含全部采样变换和重新归一化。
2. 在前向重放和反向传播期间固定当前参数值。采用目标策略的位置、SWA 和执行约定，并启用参数梯度。
3. 重算已知历史的编码器表示。从 $H_0 = (s_*, \emptyset)$ 开始，经过全部提示 token，重建递归输出与每个解码器 SWA 缓存。
4. 按顺序重放后续 token。在处理每个已采样动作之前，从前一状态读取该动作在当前策略下的对数概率。若 EOS 被采样，也应计入。处理夹在动作之间、后续预测所需的全部外部 token。
5. 根据动作对数概率、奖励或优势，以及所需的行为比率，构造选定的 RL 损失。动作因子仅作用于策略采样的 token。
6. 反向传播并更新参数；在下次精确的当前策略重放前，使依赖参数的编码器和解码器缓存失效。

动作掩码选择损失项，同时保留经过用户与工具 token 的梯度。重放全部助手回合及其间的外部上下文。在 `no_grad` 下重放提示，或分离提示 KV，可保持前向值不变，但会丢失完整 BPTT 所需的梯度。

重要性权重必须满足式 (5.5) 所述的支持集条件。

确定性的前向计算（例如关闭 dropout）可为每个动作给出可复现的概率。对于随机策略，应说明动作概率是以内部随机性为条件，还是对该随机性取平均。一次重放只评估某个具体实现；边缘概率需要对该随机性求平均。

B 因果性与梯度路径

命题 B.1 (因果性). 在确定性执行下，若编码器是因果的，编码器记忆限制为当前前缀，且解码器 SWA 是因果的，则 $H_t = (s_t, C_t^D)$ 仅依赖 $x_{1:t}$ 和参数（包括 s_* ）。

证明. 编码器的因果性意味着各 e_j 仅依赖 $x_{1:j}$ ，因此 $M_{\leq t}$ 仅依赖 $x_{1:t}$ 。初始状态不包含未来 token 的信息。假设 H_{t-1} 仅依赖 $x_{1:t-1}$ 。下次更新使用该状态、 e_t 和 $M_{\leq t}$ 。在每个解码器层，当前位置的 KV 由因果可用的层输入形成，SWA 不读取大于 t 的位置。追加当前 KV 和淘汰旧条目不会引入未来信息。因此， s_t 和 C_t^D 均仅依赖 $x_{1:t}$ ，归纳完成。□

固定教师强制得到的编码器特征，将解码器缓存表示为固定槽位，并在尚未填满时使用有效性掩码，定义

$$\mathcal{J}_t = \frac{\partial H_t}{\partial H_{t-1}}. \quad (\text{B.1})$$

则

$$\frac{\partial H_t}{\partial H_j} = \mathcal{J}_t \mathcal{J}_{t-1} \cdots \mathcal{J}_{j+1}, \quad j < t, \quad (\text{B.2})$$

其中

$$\mathcal{J}_t = \begin{pmatrix} \frac{\partial s_t}{\partial s_{t-1}} & \frac{\partial s_t}{\partial C_{t-1}^D} \\ \frac{\partial C_t^D}{\partial s_{t-1}} & \frac{\partial C_t^D}{\partial C_{t-1}^D} \end{pmatrix}. \quad (\text{B.3})$$

这个分块雅可比矩阵同时包含经过递归输出与解码器 KV 的路径。

编码器记忆保留过去编码器输出的信息；解码器 SWA 则可直接访问近期的解码器投影。较旧的解码器条目会被淘汰，但它们仍可通过淘汰前读取过它们的状态或激活影响后续计算。

完整的参数导数还包含编码器特征、记忆、各次状态转移和解码器 KV 投影对参数的依赖。令 B_T^E 表示回复续接所需的全部编码器侧边界张量，包括编码器续接 KV 和编码器生成的交叉注意力记忆。对于回复损失 $\ell(\Theta, H_T(\Theta), B_T^E(\Theta))$ ，链式法则给出

$$\frac{d\ell}{d\Theta} = \frac{\partial \ell}{\partial \Theta} + \frac{\partial \ell}{\partial H_T} \frac{\partial H_T}{\partial \Theta} + \frac{\partial \ell}{\partial B_T^E} \frac{\partial B_T^E}{\partial \Theta}. \quad (\text{B.4})$$

第一项将边界参数视为固定值，其余项计入这些边界张量的前缀计算。特别地，

$$\frac{\partial \ell}{\partial H_T} \frac{\partial H_T}{\partial \Theta} = \frac{\partial \ell}{\partial s_T} \frac{\partial s_T}{\partial \Theta} + \frac{\partial \ell}{\partial C_T^D} \frac{\partial C_T^D}{\partial \Theta}. \quad (\text{B.5})$$

即使前向概率不变，分离 s_T 、解码器 KV 或编码器边界张量，也会从完整梯度中移除相应项。

C 缓存状态何时可以复用

当保存的前缀与当前参数、已处理 token、位置、初始化、SWA 窗口及策略执行设置一致时，可以用它替代前向重放。需要保存递归输出、每个解码器 SWA 缓存、编码器续接状态、编码器记忆及相关元数据。随机计算按策略的规定处理。

已经分离梯度的缓存可能具有正确数值，却不包含训练所需的计算图。完整 BPTT 需要保留或重算前缀计算图，使梯度能够到达每个依赖参数的边界张量。

一次更新内的激活检查点以相同参数和随机状态重算激活。可变缓存的实现必须在重算时恢复保存的内容，并保留反向传播仍需使用的激活。

截断 BPTT 则保留数值，同时切断选定的梯度依赖。完整解码器状态的分离为

$$\tilde{H}_t = (\text{stopgrad}(s_t), \text{stopgrad}(C_t^D)). \quad (\text{C.1})$$

仅分离 s_t 会留下可能经过解码器 KV 的路径；仅分离解码器 KV 则会留下经过递归输出的路径。编码器侧边界张量还可能提供跨越同一边界的其他路径。

SWA 淘汰使旧 KV 不再进入未来的注意力窗口。完整 BPTT 仍会对之前使用过这些条目的计算求导，因此训练所需的激活存储可能超过推理缓存本身。

权重更新后，应在下一次精确训练计算之前，重算前缀数值及其梯度图。

D 实验细节

D.1 参数量

表 3 列出不重复计数的可训练参数量。每个 RLT 解码器模块同时具有 SWA 和编码器记忆交叉注意力；每个 Transformer 模块只有一个注意力子层。RLT 8+0 在每个 token 处应用可学习的状态投影和门控融合。

表 3 八层模型的不重复可训练参数量。同一模型在六个任务中的规模相同。

Model	Parameters (M)	Relative to GPT 8
RLT 4+4	28.73	1.135×
RLT 5+3	28.20	1.114×
RLT 6+2	27.68	1.093×
RLT 7+1	27.15	1.073×
RLT 8+0	26.10	1.031×
Transformer 8	25.31	1.000×

D.2 RLT w/o feedback 的实现

rlt_parallel_tiny 对照在采集快照后，由 [nanogptpro-dev PR #164](#) 合并。它采用不共享权重的 4+4 编码器与解码器分配、宽度 512、四个注意力头、FFN 宽度 1,365、词表容量 277、大小为八的 SWA 窗口、一个编码器记忆组，以及宽度 μP 。它共有 27,938,944 个不重复参数。

该对照从递归 RLT 4+4 中移除可学习初始状态、状态归一化、融合门控和反馈投影。它保留因果编码器注意力、解码器 SWA，以及仅关注前缀的编码器记忆注意力。已知位置在每个解码器层内并行运行，训练采用完整反向传播，不使用 TBPTT。生成阶段维护 KV 缓存并按顺序输出 token，不把前一个最终隐藏状态反馈到下一个解码器输入。两个模型使用由各自架构标识区分的独立检查点格式。

D.3 种子聚合与指标定义

令 $a_{m,s,t}$ 为模型 m 、初始化种子 s 在优化器步骤 t 的验证准确率。对于奇偶校验，报告

$$\bar{a}_{m,t} = \frac{1}{3} \sum_{s \in \{42,43,44\}} a_{m,s,t}, \quad SD_{m,t} = \sqrt{\frac{1}{2} \sum_{s \in \{42,43,44\}} (a_{m,s,t} - \bar{a}_{m,t})^2}. \quad (D.1)$$

仅当三个种子在同一步均有数值时才计算均值。缺失检查点保持缺失，因此每条奇偶校验曲线结束于其各种子共有的最后一步，并始终使用 $n = 3$ 。表 1 使用各任务中六条模型曲线共有的最新步骤。SD 描述固定训练数据流与验证集上的初始化变异。

奇偶校验和模五运算每个样本只监督一个最终标签。先合并其三个大小相同的验证组，再跨种子求平均。 S_5 在训练时监督每个前缀；最终状态准确率仅统计每条序列的最后一个状态。表 4 还报告正确前缀状态比例的均值。加法准确率要求贪心生成的答案和 EOS 完全正确，其损失则为按样本归一化的教师强制交叉熵。

表 4 固定 S_5 验证集上的最终状态 / 前缀 token 准确率 (%), 第 1,000 步, 种子 42。

Task	RLT 4+4	RLT 5+3	RLT 6+2	RLT 7+1	RLT 8+0	GPT 8
S_5 , swaps	100.00 / 100.00	99.61 / 99.80	95.70 / 99.08	89.84 / 98.73	86.33 / 98.66	89.84 / 98.75
S_5 , standard	1.17 / 3.99	3.52 / 7.82	1.95 / 7.65	1.56 / 7.06	1.17 / 4.17	1.17 / 4.00

D.4 快照复现

形式任务的训练数据种子为 20260914, 验证种子为 20260915, 单独配置的测试集种子为 20260916。加法使用数据种子 42。各模型的训练样本与留出集固定; 奇偶校验的重复运行仅改变初始化。全部已测量的运行均使用基于上游提交 d1a4516 的实现。

文件 `data/experiments-depth8/manifest.json` 和 `configs.json` 记录来源与语料指纹、运行标识、模型设置、时间戳和进度。验证记录、完整训练历史与共同步骤的聚合结果保存在同一目录下。脚本 `scripts/plot_depth8.py` 用于重新生成图表。

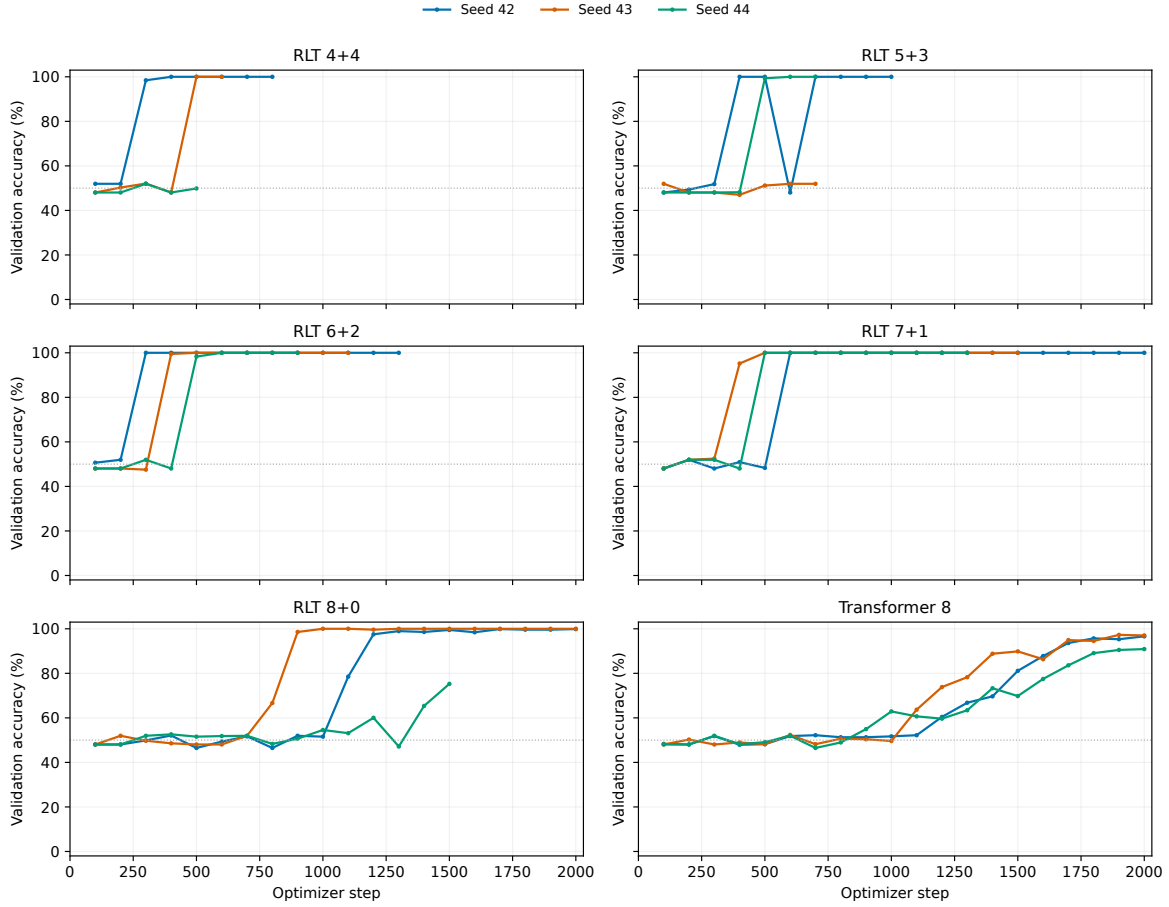


图 10 各初始化种子的奇偶校验准确率。曲线包含三种种子均值曲线终点之后的检查点, 并终止于各次运行的最新验证。所有运行使用相同的训练数据流与验证集。